# JAX

# MASTER DEVELOPER SPECIFICATION

Private Persistent Personal AI • Control Plane • Memory System • Multi-Agent Orchestrator

*“Turn Your Chaos Into Code.”*

Canonical Architecture • September 27, 2026

Prepared for a development team receiving JAX with no prior context

# 1. Executive Definition

> **JAX is Jason Manuel's private, persistent personal AI operating layer. It is not merely a chatbot, not merely an autonomous agent, and not a replacement for Git, Supermemory, Claude Code, Hermes, or other tools. JAX is the control plane that gives those systems continuity, policy, memory, synchronization, and a consistent relationship to Jason's goals.**

The simplest definition for a developer is:

```
JAX = Personal Context + Persistent Memory + Project State + Tool Orchestration + Multi-Device Sync + Agent Control + Policy + Audit
```

JAX should make it possible for Jason to stop work on one machine, move to another, select the same project, and continue from the same verified technical and conceptual state without relying on the previous LLM conversation.

## 1.1 Canonical ownership boundaries

| Domain | System of record | Why |
|---|---|---|
| Source code | Git/GitHub | Files, commits, branches, tags, releases, code history. |
| Durable personal/project context | Supermemory | Semantic context, goals, decisions, preferences, summaries, long-lived knowledge. |
| Operational state | JAX database | Current tasks, active sessions, machine state, permissions, audits, handoffs, device registry. |
| Local execution | Claude Code / Hermes / Codex-style agents / local tools | Editing, testing, browser use, code generation, task execution. |
| Human authority | Jason | Final priorities, value judgments, consequential approvals, identity, and goals. |

## 1.2 What JAX is not

- Not a hidden autonomous entity with independent authority over Jason.
- Not a system that stores every raw keystroke, screen frame, microphone sample, or password.
- Not a replacement for deterministic state systems such as Git, databases, calendars, or task stores.
- Not a single-model product. The model provider is replaceable.
- Not dependent on one coding agent. Claude, Hermes, future agents, and local models are adapters under JAX.
- Not allowed to claim that a project is synchronized because an LLM says it is. Synchronization must be mechanically proven.

# 2. Identity, Origin, and Evolution

JAX began as a COAI “Sentinel” concept: a tactical growth and diagnostic identity designed to find business problems, hunt revenue leaks, and operate in “Cipher Mode.” Earlier experiments framed JAX as a local business partner, sometimes with ChromaDB/Firebase memory concepts, lead-hunting loops, browser automation, and self-analysis. Those experiments are historical inputs, not the final architecture.

> **Current canonical direction: JAX has evolved from a COAI-specific Sentinel into Jason’s private personal AI layer across laptop, work desktop, and phone. Business capabilities remain modules inside JAX; they are no longer the whole identity.**

## 2.1 Stable identity traits to preserve

- Name: JAX.
- Core motto: “Turn Your Chaos Into Code.”
- Direct, analytical communication; no unnecessary filler.
- Operates as a strategic systems partner rather than a decorative assistant.
- Can challenge contradictions using evidence and previously declared priorities.
- Must remain subordinate to explicit permissions and Jason’s authority.
- Should know Jason deeply without pretending to be Jason.

## 2.2 Personal Operating System relationship

JAX should load Jason’s Personal Operating System as high-level policy context. It is not a command to moralize. It provides declared priorities, drift signals, and a decision framework. The core relationship is “mirror with memory”: JAX remembers what Jason said mattered and can show when actual activity diverges.

> Principle: JAX may surface a contradiction; JAX may not seize authority because of that contradiction.

# 3. Primary Product Goals and Use Cases

## Cross-device continuity

Move between laptop, work desktop, and phone without losing project state, context, decisions, or handoff information.

## Project control plane

Verify Git state, branch, remote, commit, local dirtiness, divergence, and required project instructions before an agent writes code.

## Persistent personal memory

Recall Jason's goals, preferences, recurring constraints, active ventures, decisions, and long-lived facts when relevant.

## Operational memory

Track current tasks, blockers, session state, project milestones, unresolved decisions, and follow-ups.

## Multi-agent orchestration

Launch or coordinate Claude Code, Hermes, Codex-style agents, research agents, QA agents, computer-use tools, and future providers through adapters.

## Decision support

Bring relevant evidence, previous commitments, opportunity cost, and unresolved risk into decisions without making personal decisions for Jason.

## Daily operating assistance

Morning brief, evening recap, reminders, project status, inbox of unresolved commitments, and cross-device handoffs.

## Research and synthesis

Collect public information, connected-source context, and project documents, then produce structured findings with provenance.

## Business execution

Support COAI, Harbison Standard, HBH, client systems, lead generation, documentation, websites, automation, and CRM workflows.

## Personal knowledge layer

Act as the interface to Jason's own knowledge, projects, decisions, files, and approved account integrations.

# 4. Target Architecture

```
DEVICE LAYER
  Laptop JAX Node
  Work Desktop JAX Node
  Phone Companion / PWA
        |
        v
JAX CORE API (FastAPI)
  - Identity / Auth
  - Project Registry
  - Session Manager
  - Policy Engine
  - Memory Router
  - Tool Registry
  - Agent Orchestrator
  - Notification Service
  - Audit Service
        |
        +--> PostgreSQL / Supabase
        +--> pgvector (project semantic memory if needed)
        +--> Supermemory (durable external context provider)
        +--> Git/GitHub
        +--> Model Providers (cloud + local)
        +--> MCP / Connectors / Browser / OS tools
```

## 4.1 Recommended implementation stack

| Layer | Recommended V1 | Notes |
|---|---|---|
| Core API | Python 3.12 + FastAPI | Strong orchestration ecosystem, subprocess control, typing, async APIs. |
| Database | PostgreSQL / Supabase | Structured state, users, devices, projects, tasks, sessions, audits. |
| Vector retrieval | pgvector | Use only where local semantic retrieval is needed; Supermemory remains external durable context. |
| Web UI | Next.js + TypeScript | Dashboard, project status, approvals, memory review, device health. |
| Windows node | Python service/CLI | Local Git inspection, process launch, filesystem events, approved desktop actions. |
| Phone | PWA first | Approvals, notifications, project summaries, voice input, status. Native Android later. |
| Local model option | Ollama or LM Studio adapter | Optional; not required for JAX identity or continuity. |
| Model abstraction | Provider interface | OpenAI/Anthropic/Google/local models selected by task policy. |
| Tool protocol | MCP-like registry/adapters | Keep tool access typed, permissioned, inspectable. |
| Background jobs | APScheduler/Celery later | V1 can use DB-backed jobs and a single worker; avoid premature distributed complexity. |

# 5. Memory Architecture - How JAX Learns

**JAX learning is memory formation and model-of-user refinement, not uncontrolled self-modification. JAX should not rewrite its core code or beliefs merely because one conversation produced a claim.**

## 5.1 Memory types

| Memory type | Purpose | Examples | Storage |
|---|---|---|---|
| Working memory | Short-lived current-session context | Current objective, open files, active branch, temporary scratchpad | Session store / cache |
| Episodic memory | What happened at a specific time | "On Sep 27 laptop sync failed because branch diverged." | Postgres + optional Supermemory summary |
| Semantic memory | Stable facts and concepts | Project architecture, user preferences, definitions | Supermemory + curated local records |
| Procedural memory | How to perform repeatable workflows | Deploy Harbison, preflight a repo, nightly report steps | Versioned playbooks in Git / DB |
| Project memory | State specific to one project | Decisions, constraints, milestones, handoffs | Git context files + DB + relevant Supermemory |
| Personal policy memory | Declared values/operating rules | Personal Operating System, JASON_CORE | Versioned canonical file + memory reference |

## 5.2 Learning pipeline

```
OBSERVATION / USER STATEMENT / TOOL RESULT
      |
      v
SOURCE + TIMESTAMP + SCOPE + PROVENANCE
      |
      v
WORKING MEMORY
      |
      v
MEMORY CANDIDATE CLASSIFIER
      |
      +--> transient -> expire
      +--> project -> project memory
      +--> durable personal -> Supermemory candidate
      |
      v
CONFIDENCE + IMPORTANCE + DUPLICATE CHECK
      |
      v
SAVE / UPDATE / SUPERSEDE / IGNORE
      |
      v
RETRIEVE ONLY WHEN RELEVANT
      |
      v
NEW EVIDENCE -> REINFORCE / CORRECT / DEPRECATE / FORGET
```

## 5.3 Memory candidate fields

```
MemoryCandidate {
 id
 subject
 statement
 category
```

```
  scope          // personal | project | client | device | temporary
  source_type    // user | file | tool | inferred
  source_ref
  created_at
  confidence     // 0.0 - 1.0
  importance     // 0 - 100
  sensitivity    // public | private | restricted | secret
  ttl
  supersedes_id
  evidence[]
}
```

## 5.4 Learning rules

- Explicit user corrections outrank inferred memories.
- One-off statements should not automatically become permanent identity traits.
- Inferences must be tagged as inference and should decay unless reinforced.
- Contradictory memories should not be silently merged. Preserve lineage and mark which version is current.
- Highly sensitive data should not be promoted into durable semantic memory unless explicitly required and safely stored.
- Passwords, recovery codes, private keys, raw authentication tokens, and payment credentials are never memory.
- Project facts should include a scope so Harbison context does not leak into unrelated projects.
- Memory retrieval should favor relevance and recency, but canonical project files override fuzzy recollection when they disagree.

# 6. Supermemory Integration

Supermemory is a connected durable context provider. It should not be treated as the only database and should not be used to synchronize source code. Its primary role is cross-session semantic continuity: who Jason is, what projects mean, important decisions, recurring preferences, and durable context.

## 6.1 V1 policy

- Read from Supermemory automatically when a project/session begins.
- Write only summarized, durable facts at session end or after explicit “remember this” actions.
- Do not mirror entire repositories or raw logs.
- Tag provenance and project scope in JAX even when Supermemory returns unstructured semantic results.
- Use JAX DB for operational truth; use Supermemory for semantic continuity.
- If Supermemory is unavailable, JAX must degrade gracefully and clearly mark external memory as unavailable.

## 6.2 Retrieval strategy

```
query_context(project, task):
  repo_context = load_git_context_files(project)
  jax_state    = query_operational_db(project)
  supermemory  = semantic_search(project + task + unresolved items)
  recent       = recent_handoffs(project)

  merged = normalize_and_rank(
     canonical_repo_context first,
     verified operational state second,
     recent handoff third,
     semantic memory fourth
  )

  return bounded_session_brief(merged)
```

# 7. Git/GitHub Control Plane

**Git is the authority for code. JAX must prove repository state before any managed coding agent writes to a project.**

## 7.1 Preflight state machine

| State | Meaning | Default JAX action |
|---|---|---|
| READY | Clean tree; local == upstream | Allow agent launch. |
| BEHIND | Clean; remote ahead; no local commits | Fetch + fast-forward-only, re-check. |
| AHEAD | Local commits not pushed | Warn; user/policy decides whether to push. |
| DIRTY | Uncommitted changes exist | Block automatic sync; create handoff/decision. |
| DIVERGED | Both sides have unique commits | Block; no automatic merge/rebase. |
| NO_UPSTREAM | Branch lacks upstream | Require setup or explicit local-only mode. |
| DETACHED | Detached HEAD | Block normal workflow until resolved. |
| REMOTE_UNREACHABLE | Cannot verify remote | Do not claim synchronization. |

## 7.2 Required commands

```
git rev-parse --show-toplevel
git remote -v
git branch --show-current
git status --porcelain=v1
git fetch --all --prune
git rev-parse HEAD
git rev-parse @{u}
git rev-list --left-right --count HEAD...@{u}
git pull --ff-only   # only when proven safe
```

## 7.3 Canonical context files inside each project

```
AGENTS.md          // instructions for coding agents
PROJECT_STATE.md   // current milestone, verified status, environment facts
DECISIONS.md       // durable architecture/product decisions
HANDOFF.md         // latest handoff and next action
TODO.md            // explicit current work queue if project uses it
.jax/project.yaml  // project id, commands, adapters, safety policy
```

# 8. Multi-Device Continuity

Each machine receives a stable local identity. JAX records where a session happened, the starting commit, ending commit, tools used, tests run, and handoff summary. Device identity is local configuration and is not stored as a shared machine secret in Git.

```
~/.jax/config.yaml
machine_id: travel-laptop
projects_root: C:\Users\blunts\Desktop\projects
api_url: https://jax-core.example
```

## 8.1 Handoff invariant

> **The next device must be able to reconstruct the project from Git + canonical context + JAX operational state + relevant Supermemory. It must never require the previous machine's conversational transcript.**

## 8.2 Phone role

- Approvals for consequential actions.
- Notifications and alerts.
- Project status and latest handoff.
- Voice/text commands.
- Reminders and daily briefs.
- Optional file/photo input.
- Not a separate uncontrolled code-editing authority in V1.

# 9. Multi-Agent Orchestration

JAX should treat coding agents and LLMs as workers with different strengths. JAX owns the session contract; the selected agent performs the task. The agent may be swapped without losing project continuity.

## 9.1 Adapter contract

```
class AgentAdapter(Protocol):
  name: str
  def available(self) -> bool: ...
  def version(self) -> str: ...
  def launch(self, cwd: str, brief: SessionBrief) -> AgentSession: ...
  def healthcheck(self) -> dict: ...
  def collect_result(self, session_id: str) -> AgentResult: ...
  def stop(self, session_id: str) -> None: ...
```

## 9.2 Initial adapters

- Claude Code adapter: repo-aware coding and reasoning.
- Hermes adapter: local agent/TUI workflows and tool execution.
- Generic command adapter: any CLI coding agent that can accept working directory and prompt/context.
- Browser/research adapter: public-web and connected-source research.
- QA adapter: tests outputs from other agents and checks against acceptance criteria.

## 9.3 Specialist roles (later)

| Role | Purpose | Autonomy |
|---|---|---|
| Coding agent | Implement scoped code tasks | L2-L3 depending policy |
| Research agent | Gather evidence and summarize | L1-L2 |
| Business agent | Prepare prospect/client analysis | L1-L2 |
| Task manager | Track commitments, blockers, follow-up | L1-L2 |
| Computer agent | Operate approved local UI workflows | L2-L3 |
| QA/security agent | Review outputs, permissions, tests | L1-L2 |
| Trend agent | Monitor defined signals and produce briefs | L1-L2 |

# 10. Permission and Autonomy Model

| Level | Name | Examples |
|---|---|---|
| L0 | Observe | Read project status, inspect Git, retrieve memory, read logs. |
| L1 | Organize | Classify files, summarize, update internal task state, prepare recommendations. |
| L2 | Prepare | Draft code patches, emails, deployment plans, commands, forms - but do not execute consequential action. |
| L3 | Execute approved/reversible | Run tests, create branches, edit approved files, open local applications, safe sync. |
| L4 | Explicit confirmation | Send messages, deploy production, delete data, purchase, account changes, merge/rebase/force operations. |

## 10.1 Hard blocks

- Password extraction or credential exfiltration.
- Disabling security controls to make automation easier.
- Continuous raw keylogging.
- Continuous raw microphone/video capture without a specific approved mode.
- Autonomous destructive Git operations.
- Autonomous purchases or financial transfers.
- Autonomous production deletion or account takeover actions.
- Unreviewed self-modification of core policy or permission code.

# 11. Context Collection and “Senses”

JAX should be context-aware without becoming a surveillance system. Prefer structured events over raw capture. Context collection is opt-in, scoped, and inspectable.

| Signal | Preferred method | Avoid |
|---|---|---|
| Active project | Git root + editor/workspace integration | Guessing from window title alone |
| File changes | Filesystem watcher with allowlist | Reading entire disk continuously |
| Browser context | Explicit browser extension/tool session | Permanent full browsing history ingestion |
| Calendar | Connector/API | Screen scraping calendar UI |
| Email | Connector search/read with permissions | Ingesting all mail permanently |
| Voice | Push-to-talk / explicit wake session | Always-on raw audio retention |
| Screen | User-triggered screenshot/Computer Use session | 24/7 screenshots |
| Processes | Local node health telemetry | Undocumented process injection |

# 12. Model Provider and Reasoning Layer

JAX identity must not depend on one model. The orchestration layer selects a model based on task type, cost, latency, privacy, and required capabilities.

```
ModelRequest {
 task_type
 sensitivity
 max_cost
 latency_class
 requires_tools
 requires_vision
 requires_long_context
 preferred_provider?
}

route(request) -> ProviderAdapter + model
```

## 12.1 Routing principles

- Use local models for privacy-sensitive lightweight tasks when adequate.
- Use stronger cloud models for complex coding/reasoning when permitted.
- Never let a model response override deterministic tool state.
- Capture provider/model/version in the audit trail for consequential work.
- Provider failure should permit fallback where policy allows.

# 13. Project Intelligence Layer

A JAX project is more than a folder. It is a tracked operating object containing code state, goals, people, constraints, current milestone, tasks, artifacts, memory references, and tool permissions.

```
Project {
 id
 name
 repo_url
 local_paths_by_device{}
 current_milestone
 status
 stakeholders[]
 constraints[]
 commands { install, dev, test, lint, build, deploy }
 allowed_agents[]
 memory_scope
 risk_level
 created_at
 updated_at
}
```

## 13.1 Project brief generated at session start

- What this project is.
- What Jason is trying to accomplish now.
- Current verified commit and branch.
- Last completed work.
- Known constraints (“preserve existing visual design,” etc.).
- Open blockers and unresolved decisions.
- Acceptance criteria for the current task.
- Relevant user/client preferences.
- Permitted tools and actions.
- Exact commands required to test the work.

# 14. JAX Daily, Drift Detection, and Personal Alignment

JAX Daily is the operating cadence that turns memory into useful continuity. It should not be a motivational monologue. It should surface reality: what changed, what is blocked, what is overdue, and what deserves attention.

## 14.1 Morning brief

- Top three outcomes for today
- Commitments due
- Project changes since last session
- Unresolved blockers
- Relevant meetings/reminders
- High-risk drift warning if present

## 14.2 Evening recap

- What was completed
- What changed in project state
- What was postponed and why
- Decisions made
- New memory candidates
- Tomorrow's first action

## 14.3 Drift triggers

| Trigger | Example JAX response |
|---|---|
| Project switching | "Three projects were started today; none reached the defined done condition." |
| Research overrun | "The decision-relevant uncertainty appears resolved; another 2 hours of research has not changed the recommendation." |
| Complexity creep | "This adds a new subsystem before the current workflow is proven." |
| Priority conflict | "This new task displaces the milestone you declared primary yesterday." |
| Financial drift | "Activity increased, but no new retained revenue or owned asset was created." |

# 15. Core Database Schema

```
users
  id, display_name, timezone, created_at

devices
  id, user_id, machine_id, name, os, last_seen_at, public_key, revoked_at

projects
  id, user_id, name, repo_url, memory_scope, status, risk_level

project_device_paths
  project_id, device_id, local_path

sessions
  id, project_id, device_id, agent_name, base_commit, final_commit, started_at, ended_at, status

handoffs
  id, session_id, summary, changed_files_json, tests_json, decisions_json, unresolved_json, next_actions_json

tasks
  id, project_id, title, status, priority, due_at, source, parent_id

memory_refs
  id, project_id, external_provider, external_id, category, confidence, provenance_json

tool_permissions
  id, user_id, project_id, tool_name, action, max_level, constraints_json

audit_events
  id, timestamp, device_id, session_id, actor, action, target, result, metadata_json
```

# 16. Core API Surface

| Method | Endpoint | Purpose |
|---|---|---|
| POST | /v1/sessions/preflight | Verify device/repo/project state. |
| POST | /v1/sessions | Start a managed JAX session. |
| POST | /v1/sessions/{id}/launch-agent | Launch selected adapter after policy check. |
| POST | /v1/sessions/{id}/verify | Run configured test/lint/build verification. |
| POST | /v1/sessions/{id}/handoff | Close session and write handoff. |
| GET | /v1/projects | Project registry/status. |
| GET | /v1/projects/{id}/context | Canonical bounded context bundle. |
| POST | /v1/memory/search | Search merged memory context. |
| POST | /v1/memory/candidates | Submit a durable memory candidate. |
| GET | /v1/approvals | Pending L4 actions. |
| POST | /v1/approvals/{id} | Approve/deny action. |
| GET | /v1/devices | Device health and sync state. |
| GET | /v1/audit | Queryable audit trail. |

# 17. Windows JAX Node

The Windows node is a lightweight local service/CLI on each PC. It is the trusted bridge between JAX Core and local Git/files/processes. It should be intentionally narrow and permissioned.

## 17.1 Responsibilities

- Report device identity and health.
- Resolve configured project paths.
- Inspect Git deterministically.
- Launch approved local agent processes.
- Run approved project commands.
- Watch allowlisted file changes.
- Collect process exit codes and logs.
- Encrypt/secure local credentials using Windows credential storage where possible.
- Support kill switch and service stop.

## 17.2 Node must not

- Expose arbitrary remote shell by default
- Scan unrelated user folders
- Read browser passwords
- Capture keyboard globally
- Run as admin unless a narrowly defined task requires elevation

# 18. User Interfaces

## 18.1 Desktop/web dashboard

- Home: Today, active projects, blockers, approvals, JAX Daily.
- Projects: READY/BEHIND/AHEAD/DIRTY/DIVERGED/BLOCKED status.
- Sessions: active agent, device, branch, commit, elapsed time, last event.
- Memory: recent durable memories, provenance, corrections, forget/supersede controls.
- Approvals: pending L4 actions with exact command/action and consequences.
- Audit: chronological tool/action log.
- Devices: laptop/work desktop/phone health, last seen, active session.
- Settings: provider routing, permissions, privacy, project roots.

## 18.2 Phone companion

- Push notifications
- Approve/deny L4 actions
- Voice/text command
- Read handoffs and summaries
- See device/project status
- Trigger safe non-destructive routines

# 19. Voice Interaction

Voice is an interface, not a separate identity. A voice session should transcribe into the same session/context system. Keep raw audio only as long as required by explicit settings.

```
Voice flow:
Push-to-talk -> speech-to-text -> intent classifier -> context retrieval -> model/tool plan -> spoken response

For actions:
spoken request -> policy evaluation -> preview -> confirmation if required -> execute -> audit -> spoken result
```

# 20. Tool Registry and MCP/Connector Architecture

Every capability JAX can invoke should be registered with a typed schema, risk level, permission requirements, and audit behavior. Do not give the model a vague “computer access” super-tool when narrower tools exist.

```
ToolDefinition {
 name
 description
 input_schema
 risk_level
 permission_level
 reversible
 network_scope
 data_classes_accessed[]
 timeout_seconds
}
```

## 20.1 Example tools

- git.inspect, git.fetch, git.fast_forward, git.diff
- files.search_project, files.read_project, files.write_project
- process.launch_agent, process.run_test
- web.search, browser.open
- calendar.read, calendar.create (separate permissions)
- email.search, email.draft, email.send (send = L4)
- crm.read, crm.create/update (project-scoped)
- notifications.push
- memory.search, memory.propose, memory.supersede

# 21. Security and Threat Model

## 21.1 Primary threats

- Prompt injection from websites, files, or tool output.
- Credential leakage into logs or memory.
- Agent writes to wrong repository/device.
- Malicious or accidental destructive Git actions.
- Unauthorized device impersonation.
- Overbroad filesystem/browser access.
- Supply-chain risk from agent plugins/MCP servers.
- Autonomous loops that continue after intended task completion.
- Cross-project data leakage.
- Stale context causing incorrect execution.

## 21.2 Required controls

- Per-device keys/tokens with revocation.
- TLS for network communication.
- OS credential store for secrets; environment variables only where necessary.
- Project allowlists and path canonicalization.
- Tool-level schemas and permission checks outside the model.
- Action previews for consequential operations.
- Immutable append-only audit events (or tamper-evident later).
- Time/iteration budgets for autonomous loops.
- Kill switch that stops local agents and background workers.
- Redaction of secrets from logs before persistence.
- Backups of Postgres and canonical project context files.

# 22. Self-Improvement Without Unsafe Self-Modification

Earlier JAX concepts included “self-fixing” or “self-evolution.” The safe production interpretation is: JAX may inspect its own code, identify defects, create a patch branch, run tests, and propose a change. It should not silently rewrite and deploy its own control/permission logic.

```
SAFE SELF-IMPROVEMENT
1. Detect defect / optimization opportunity
2. Create issue or proposal
3. Create isolated branch
4. Generate patch
5. Run tests + security checks
6. Produce diff and rationale
7. Require approval for core/policy changes
8. Merge/deploy through normal pipeline
9. Record audit event
```

# 23. Failure Handling and Recovery

| Failure | JAX response |
|---|---|
| Supermemory unavailable | Use Git context + JAX DB; mark semantic external memory unavailable. |
| GitHub unavailable | Operate read-only/local as configured; never claim remote sync. |
| Agent crashes | Capture exit/logs; keep session state; allow resume or adapter switch. |
| JAX Core unavailable | Local node allows only predefined offline-safe actions; queue state for later sync. |
| Device disappears mid-session | Mark session interrupted; next device sees unresolved handoff. |
| Memory contradiction | Surface both with provenance; prefer explicit correction/canonical source. |
| Tool timeout | Stop tool, preserve logs, require retry decision. |
| Permission ambiguity | Fail closed for consequential actions. |

# 24. Offline and Degraded Modes

- Offline local chat/model can operate if a local model is installed.
- Local project context and Git history remain accessible.
- Remote synchronization status becomes “unverified,” not “ready.”
- New durable memories may be queued locally for later reconciliation.
- Actions requiring connected services are disabled or queued with explicit status.
- Phone approval cannot be assumed if phone/core is unreachable.

# 25. Recommended JAX Repository Structure

```
jax/
  pyproject.toml
  README.md
  .env.example
  docs/
    architecture.md
    threat-model.md
    permissions.md
    memory-model.md
  src/jax/
    api/
    core/
      control_plane.py
      session_manager.py
      policy_engine.py
      project_registry.py
    git/
      service.py
      state.py
    memory/
      router.py
      supermemory.py
      local_store.py
      candidate_pipeline.py
    agents/
      base.py
      claude.py
      hermes.py
      generic_cli.py
    tools/
      registry.py
      permissions.py
    devices/
      registry.py
      protocol.py
    audit/
      logger.py
    models/
      router.py
      providers/
    notifications/
    daily/
  node_windows/
  web/
  phone_pwa/
  tests/
    unit/
    integration/
    e2e/
```

# 26. Developer Build Order - Start to Finish

## Phase 0 - Specification freeze

Confirm project vocabulary, device names, permission levels, canonical context files, and initial agents.

## Phase 1 - Core repo + database

FastAPI skeleton, Postgres schema, migrations, logging, config, auth foundations.

## Phase 2 - Git state engine

Implement deterministic preflight, state machine, tests for READY/BEHIND/AHEAD/DIRTY/DIVERGED.

## Phase 3 - Project registry

Projects, commands, device paths, constraints, risk level, canonical context loader.

## Phase 4 - Windows laptop node

Laptop-first local service/CLI, device identity, project discovery, approved subprocess execution.

## Phase 5 - Session + handoff engine

Start/close session, base/final SHA, changed files, tests, decisions, next actions.

## Phase 6 - Supermemory adapter

Read-only retrieval first; normalization, provenance, bounded context bundle.

## Phase 7 - Memory candidate pipeline

Explicit remember, session summaries, confidence/scope, supersede/forget workflow.

## Phase 8 - Agent adapters

Claude first, Hermes second, generic CLI third.

## Phase 9 - Verification hooks

Per-project test/lint/build commands and pass/fail gates.

## Phase 10 - Dashboard

Project/device/session/memory/approval/audit UI.

## Phase 11 - Work desktop node

Install same node, reconcile paths, test cross-device handoff deliberately.

## Phase 12 - Phone PWA

Approvals, alerts, status, summaries, voice/text input.

## Phase 13 - JAX Daily + reminders

Morning/evening brief, drift detector, follow-up queue.

## Phase 14 - Tool registry + connectors

Add browser, calendar, email, CRM, etc. one permissioned integration at a time.

## Phase 15 - Hardening

Threat-model review, secret redaction, backups, kill switch, failure drills, telemetry.

## Phase 16 - Specialist agents

Research/business/QA/trend/computer agents under the same control plane.

# 27. Acceptance Tests

- Laptop is 3 commits behind and clean -> JAX fetches, fast-forwards only, verifies matching SHA, then allows agent launch.
- Laptop has uncommitted edits -> JAX blocks automatic synchronization and shows exact changed files.
- Laptop and work desktop created different commits -> JAX detects divergence and refuses to guess.
- Claude session ends -> JAX records final SHA, changed files, tests, decisions, unresolved work, and next action.
- User moves to work desktop -> JAX reconstructs project from verified code + handoff + memory without old Claude chat.
- Supermemory returns an outdated fact contradicted by PROJECT_STATE.md -> canonical project state wins and contradiction is logged.
- User says “remember this” -> JAX proposes durable memory with source, scope, and sensitivity; no secret is stored.
- Agent requests force push -> JAX requires L4 approval.
- Agent crashes -> session remains recoverable and another adapter can receive the same brief.
- Phone approval denied -> action does not execute.
- JAX Core loses internet -> device does not claim GitHub is synchronized.
- Prompt injection in a web page asks JAX to reveal secrets -> tool boundary prevents secret access.

# 28. Product Milestones

| Release | Definition |
|---|---|
| JAX V1 - Continuity | Laptop + work desktop; deterministic Git preflight; project registry; session handoffs; Supermemory retrieval; Claude/Hermes adapters; audit; basic dashboard. |
| JAX V2 - Personal Operating Layer | Phone PWA; JAX Daily; durable memory candidate pipeline; reminders; richer project/task intelligence; approval workflow; model routing. |
| JAX V3 - Delegated Operations | Permissioned business/research/computer agents; calendar/email/CRM connectors; proactive monitoring; advanced QA/security; limited safe automation loops. |
| Long-term | Portable identity/context layer across devices and models, with strong provenance, explicit permissions, and replaceable providers. |

# 29. Definition of Done for the First Real JAX

> **JAX V1 is successful when Jason can stop work on one computer, move to another, select the same project, and JAX can prove which commit is canonical, synchronize safely when possible, refuse when unsafe, reconstruct the project's current objective and constraints, launch the chosen coding agent with the same bounded context, verify the result, and produce a durable handoff - without depending on the previous agent chat.**

That is the foundational problem to solve before adding flashy autonomy. If continuity is unreliable, more agents create more desynchronization rather than more capability.

# 30. Developer Quick-Start Checklist

1. Read this entire specification and the Personal Operating System/JASON_CORE before implementation.
2. Implement Git state engine and tests before any LLM integration.
3. Install JAX on the travel laptop first and use it as the reference node.
4. Use a fresh test repository to simulate dirty/ahead/behind/diverged states.
5. Implement project registry and HANDOFF.md lifecycle.
6. Add Supermemory read-only retrieval and context compilation.
7. Add Claude Code adapter; verify session continuity across restart.
8. Add Hermes adapter; explicitly test abnormal TUI/process termination behavior.
9. Install work desktop node; run controlled cross-device handoff tests.
10. Only after the above is stable, add phone approvals, Daily briefs, and more autonomous tools.

# Appendix A - JASON_CORE for JAX

IDENTITY
Jason Manuel is a systems-oriented builder becoming an architect and steward. His recurring competency is architecture under chaos: finding structure across disconnected domains and turning disorder into repeatable systems.

PRIMARY OPERATING PRINCIPLE
I turn chaos into structure. When chaos appears, I become the structure.

PRIORITY ORDER
1. Reality and integrity.
2. Health and human responsibilities.
3. Current primary economic engine.
4. Completion and operation of existing systems.
5. New exploration within an explicit time budget.

EXECUTION RULES
- Finish before expanding.
- Evidence outranks narrative.
- Every major project should create an owned asset.
- New priorities must explicitly displace old priorities.
- Curiosity is valuable but does not receive unlimited command authority.
- Measure shipped outcomes, retained value, and compounding assets rather than activity.

JAX ROLE
JAX is a mirror with memory. It remembers, organizes, monitors, challenges contradictions, and reduces administrative entropy. It does not determine Jason's values, make itself an authority over Jason, or take uncontrolled autonomous action.

DRIFT TRIGGERS
Flag complexity creep, repeated project switching, research beyond decision value, tool switching without workflow diagnosis, unfinished commitments, and activity that does not create revenue or an owned asset.

CHALLENGE FORMAT
State the conflict without drama. Show the evidence. Identify what would be displaced. Offer: brief exploration, explicit reprioritization, or return to execution. Jason makes the decision.

LONG-RANGE STANDARD
Build more than you destroy. Become more stable as you become more capable. Leave systems, opportunities, knowledge, and people stronger than you found them.

# Appendix B - Session Brief Template

```
SESSION BRIEF
Project:
Device:
Agent:
Timestamp:

VERIFIED REPOSITORY STATE
Branch:
Local HEAD:
Remote HEAD:
Sync state:
Dirty files:

CURRENT OBJECTIVE
<one clear outcome>

CURRENT PROJECT STATE
<short verified summary>

CONSTRAINTS
- ...

LAST HANDOFF
<what changed, what remains>

RELEVANT DURABLE CONTEXT
<bounded Supermemory + canonical context>

ACCEPTANCE CRITERIA
- ...

PERMITTED ACTIONS
- ...

REQUIRES CONFIRMATION
- ...

TEST COMMANDS
- ...
```

# Appendix C - Handoff Template

```
HANDOFF
Session ID:
Machine:
Agent:
Base commit:
Final commit:

COMPLETED
- ...

CHANGED FILES
- ...

TESTS / VERIFICATION
- command -> result

DECISIONS
- ...

UNRESOLVED
- ...

NEXT ACTION
1. ...

RISKS / WARNINGS
- ...
```

# Appendix D - Legacy Concepts and Canonical Status

| Earlier concept | Current status |
|---|---|
| COAI Sentinel / revenue hunter | Preserved as an optional business persona/module; no longer the whole product. |
| Firebase as the “brain” | Not canonical. PostgreSQL/Supabase + Supermemory is the current recommended design. |
| ChromaDB local memory | Optional/legacy. Prefer pgvector if local semantic storage is needed. |
| Always-on 90-second autonomous loops | Not canonical. Use explicit scheduled jobs with budgets, audit, and permissions. |
| Self-fixing code automatically | Reinterpreted as propose/test/review pipeline; core changes require approval. |
| “God-mode” autonomy | Rejected as an engineering requirement. Capability is constrained by permission levels and policy. |
| Jarvis-style assistant identity | Not desired. JAX is a systems partner/control layer, not a butler persona. |

# 31. Final Engineering Principle

> **Build JAX as a deterministic control system around nondeterministic intelligence. The models may change. The coding agents may change. Devices may change. The durable value of JAX is that Jason's projects, decisions, permissions, context, and continuity survive those changes.**

The first milestone is therefore not "make JAX sound impressive." It is: make JAX trustworthy enough that two computers cannot silently become two different realities again.